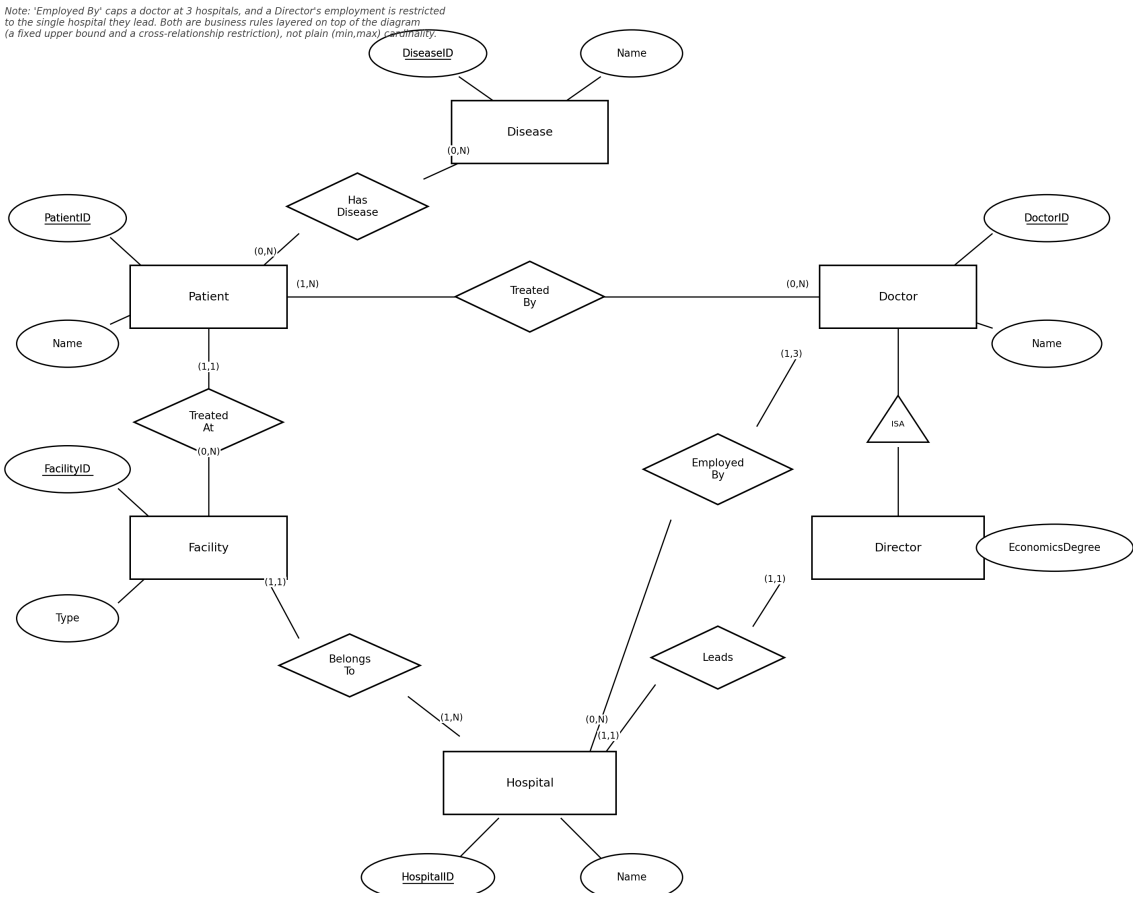
DBMS Lab 2: ER Modeling and ER-to-Relational Mapping

Question 1: Hospitals, Facilities, Doctors and Patients

This diagram is built from scratch to satisfy the business rules given in the assignment. It uses a Doctor-to-Director specialization, since a director is simply a doctor who takes on an extra role with its own attribute and its own participation constraints.



Original ER diagram for Question 1.

Entities and keys

- **Patient**(PatientID, Name) - key: PatientID.
- **Disease**(DiseaseID, Name) - key: DiseaseID.
- **Doctor**(DoctorID, Name, Specialization) - key: DoctorID.
- **Director** ISA Doctor, adds (EconomicsDegree) - inherits DoctorID as its key.
- **Facility**(FacilityID, Type) - key: FacilityID.
- **Hospital**(HospitalID, Name) - key: HospitalID.

Relationships and cardinalities

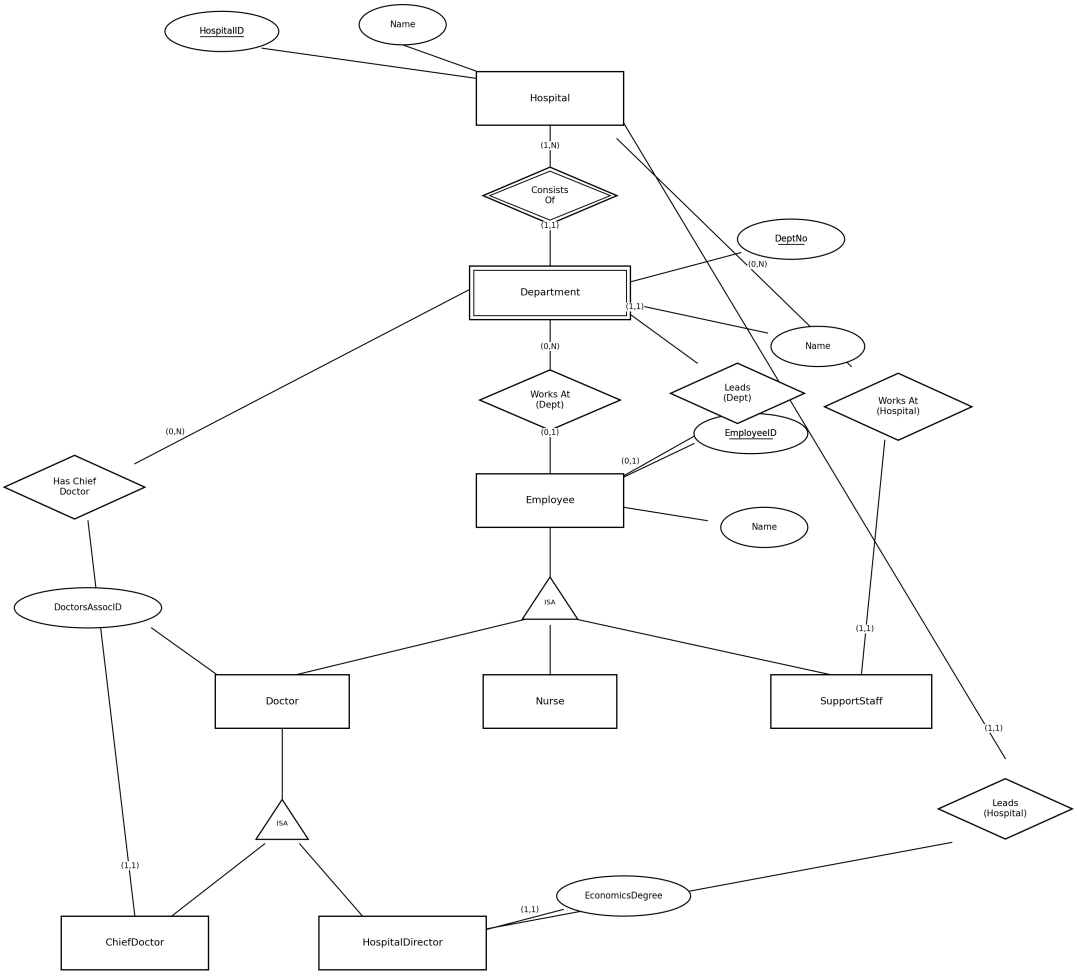
- **HasDisease**: Patient (0,N) - Disease (0,N). A patient can have several diseases at once, or none right now; a disease can currently have no patients at all.

- **TreatedBy**: Patient (1,N) - Doctor (0,N). Every patient is treated by at least one doctor, possibly several; a doctor may have any number of patients, including zero.
- **TreatedAt**: Patient (1,1) - Facility (0,N). Each patient is assigned to exactly one facility; a facility may currently hold no patients ('might be empty').
- **BelongsTo**: Facility (1,1) - Hospital (1,N). Each facility belongs to exactly one hospital; every hospital is assumed to have at least one facility.
- **EmployedBy**: Doctor (1,3) - Hospital (0,N). A doctor works at one to three hospitals; a hospital may employ any number of doctors.
- **Leads**: Director (1,1) - Hospital (1,1). Every hospital has exactly one director, and a director leads exactly one hospital.

Two business rules fall outside what plain (min,max) cardinality can express, and are noted on the diagram rather than forced into a cardinality label: the upper bound of exactly three hospitals per doctor is a fixed numeric cap, not a range, and the rule that a director's own employment is confined to the single hospital they lead is a restriction on one relationship (EmployedBy) that depends on the outcome of a different relationship (Leads). Both would need a CHECK constraint or trigger in the relational implementation.

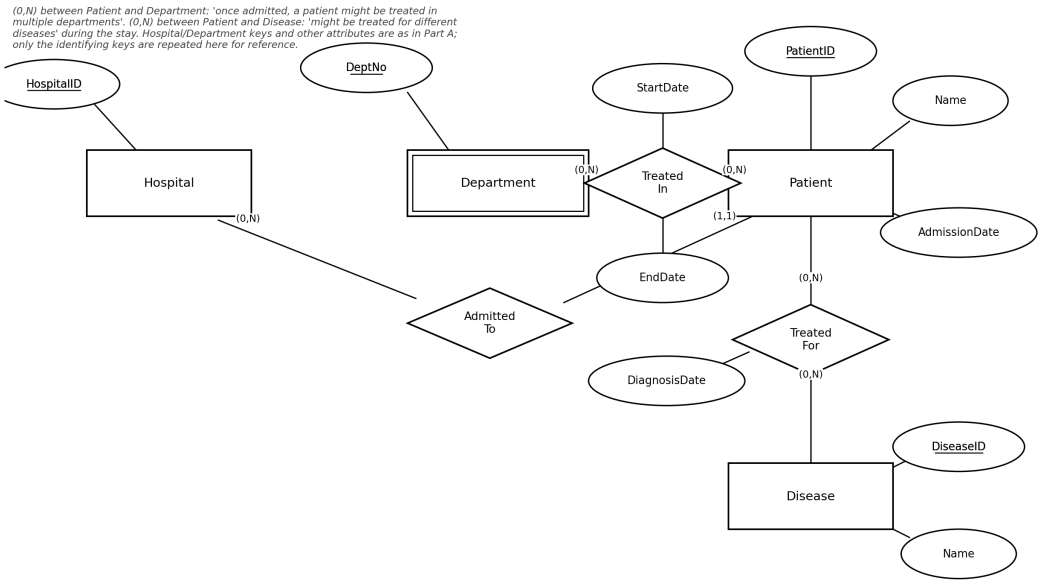
Question 2: Hospital Staffing and Patient Care

Split across two diagrams for readability: staffing and leadership structure first, then how patients move through departments and diseases during a stay. Department is modelled as a weak entity, since a department number is only meaningful within its own hospital.



Note: policy (not expressible as plain cardinality) - a department's leader must be one of its own chief doctors if at least one exists, otherwise any employee is 'commissioned' as leader instead. Chief doctors, department leaders and the hospital director all hold a medical (director: economics) degree and work at only this one hospital.
'Works At (Dept)' is shown at the Employee level since both Doctor and Nurse participate; it is populated (exactly one department) only for Doctor/Nurse instances and never for SupportStaff, which instead has mandatory 'Works At (Hospital)'.

Question 2, Part A: staffing and leadership.



Question 2, Part B: patient admission and treatment.

Entities and keys

- **Hospital**(HospitalID, Name) - key: HospitalID.
- **Department**(DeptNo, Name) - weak entity, key: (HospitalID, DeptNo), owned by Hospital through Consists Of.
- **Employee**(EmployeeID, Name) - key: EmployeeID.
- **Doctor** ISA Employee, adds (DoctorsAssocID).
- **Nurse** ISA Employee - no extra attributes beyond Employee.
- **SupportStaff** ISA Employee - no extra attributes beyond Employee.
- **ChiefDoctor** ISA Doctor - no extra attributes; existence in this subclass is what makes a doctor eligible to head a department's chief-doctor roster.
- **HospitalDirector** ISA Doctor, adds (EconomicsDegree).
- **Patient**(PatientID, Name, AdmissionDate) - key: PatientID.
- **Disease**(DiseaseID, Name) - key: DiseaseID.

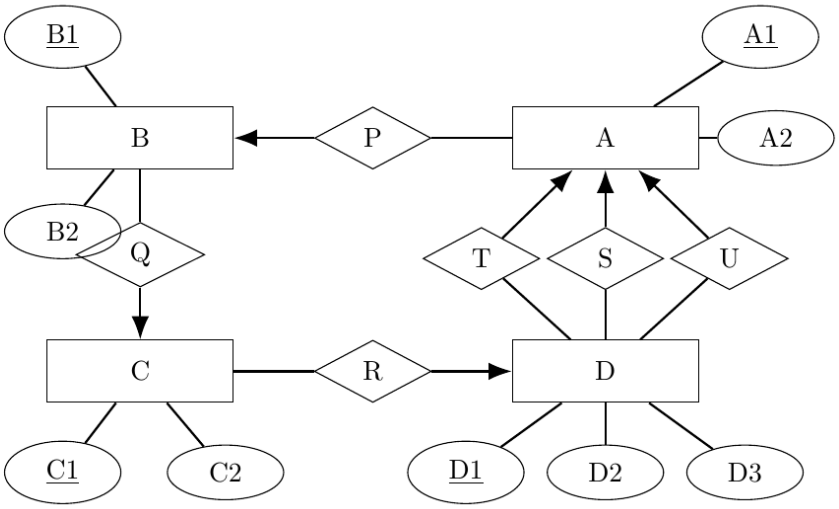
Relationships and cardinalities

- **ConsistsOf** (identifying): Hospital (1,N) - Department (1,1). Every department belongs to exactly one hospital; a hospital consists of at least one department.
- **WorksAt(Dept)**: Employee (0,1) - Department (0,N). Drawn at the Employee level because both Doctor and Nurse participate; a given employee works at zero departments (SupportStaff) or exactly one (Doctor/Nurse), and a department has any number of doctors and nurses.
- **WorksAt(Hospital)**: SupportStaff (1,1) - Hospital (0,N). Support staff attach directly to the hospital rather than to a department.
- **HasChiefDoctor**: Department (0,N) - ChiefDoctor (1,1). A department has any number of chief doctors; each chief doctor is chief at exactly one department.
- **Leads(Dept)**: Department (1,1) - Employee (0,1). Every department has exactly one leader, who is some employee (of any subtype); a given employee leads at most one department.

- **Leads(Hospital)**: Hospital (1,1) - HospitalDirector (1,1). Every hospital has exactly one director, and every director leads exactly one hospital.
- **AdmittedTo**: Patient (1,1) - Hospital (0,N). A patient's overall stay belongs to exactly one hospital.
- **TreatedIn** (with StartDate, EndDate): Patient (0,N) - Department (0,N). A patient may be treated in several departments over the course of one admission.
- **TreatedFor** (with DiagnosisDate): Patient (0,N) - Disease (0,N). A patient may be treated for several diseases during the same stay.

Business rule noted outside the cardinalities: a department's leader must be one of its own chief doctors whenever at least one exists; only when a department has no chief doctor at all is a 'commissioned' leader of any employee type allowed. This is a conditional cross-relationship constraint (it depends on how many rows HasChiefDoctor has for that department), so it cannot be written as a plain (min,max) pair on Leads(Dept) and is instead documented as a rule the application or a trigger must enforce. Every chief doctor, every department leader and the hospital director carry a required degree (medical, or economics for the director) and single-hospital employment; this is the same kind of business-rule restriction as the director's employment cap in Question 1.

Question 3: ER-to-Relational Mapping (Given Diagram)



Given ER diagram (arrow notation: an arrowhead points to the entity with maximum cardinality 1).

Reading the arrows

- **P** (A-B): arrow into B → each A relates to at most one B; a B may relate to many A's. Many-to-one from A to B.
- **Q** (B-C): arrow into C → each B relates to at most one C; a C may relate to many B's. Many-to-one from B to C.
- **R** (C-D): arrow into D → each C relates to at most one D; a D may relate to many C's. Many-to-one from C to D.
- **T, S, U** (three separate relationships, each between D and A): arrows into A → each D relates, via each of T, S and U independently, to at most one A. Three separate many-to-one relationships from D to A.

No relationship here is many-to-many: every one of P, Q, R, T, S and U is many-to-one. That means none of them needs a table of its own - each can be folded into the entity on the 'many' side as a foreign key. Since the diagram gives no double lines for total participation, participation is taken to be partial (0 as the minimum) on every side, so every foreign key introduced below is nullable.

Minimal relational schema (4 relations, no separate relationship tables)

A(A1, A2, P_B)

B(B1, B2, Q_C)

C(C1, C2, R_D)

D(D1, D2, D3, T_A, S_A, U_A)

- **A**: primary key A1. Foreign key P_B → B(B1), nullable (P is optional on the A side).
- **B**: primary key B1. Foreign key Q_C → C(C1), nullable.
- **C**: primary key C1. Foreign key R_D → D(D1), nullable.
- **D**: primary key D1. Three separate foreign keys, all → A(A1): T_A, S_A, U_A, each nullable. Three columns are needed, not one, because T, S and U are three distinct relationships - a single D row can be linked to a different A through each of them.

Why A and D don't also get a foreign key: A is the 'one' side of P (so B holds that key, not A), and the 'one' side of T/S/U (so D holds those keys, not A). D is the 'one' side of R (so C holds that key, not D). A table only receives a foreign key when it sits on the 'many' side of a relationship.

Question 4: Relational Schemas for Questions 1 and 2

Part (a) below applies the direct, unoptimised transformation rule (every entity becomes a table; every relationship - including many-to-one ones - also becomes its own table referencing the participants). Part (b) then folds every many-to-one and one-to-one relationship into the entity on the 'many' (or either) side as a foreign key, leaving separate tables only for genuine many-to-many relationships and relationships that carry their own descriptive attributes spanning more than two participants. This is the same two-pass approach used in Question 3, now applied to the diagrams designed for Questions 1 and 2.

Question 1 - (a) direct transformation (one table per relationship)

Patient(PatientID, Name)

Disease(DiseaseID, Name)

Doctor(DoctorID, Name, Specialization)

Director(DoctorID, EconomicsDegree) - FK DoctorID → Doctor(DoctorID)

Facility(FacilityID, Type)

Hospital(HospitalID, Name)

HasDisease(PatientID, DiseaseID) - FKs → Patient, Disease

TreatedBy(PatientID, DoctorID) - FKs → Patient, Doctor

TreatedAt(PatientID, FacilityID) - FK PatientID → Patient (also the PK, since each patient has one facility), FK FacilityID → Facility

BelongsTo(FacilityID, HospitalID) - FK FacilityID → Facility (also the PK), FK HospitalID → Hospital

EmployedBy(DoctorID, HospitalID) - FKs → Doctor, Hospital

Leads(HospitalID, DoctorID) - FK HospitalID → Hospital (also the PK), FK DoctorID → Director

Question 1 - (b) minimal transformation

Patient(PatientID, Name)

Disease(DiseaseID, Name)

Doctor(DoctorID, Name, Specialization)

Director(DoctorID, EconomicsDegree, Leads_HospitalID NOT NULL) - FKs: DoctorID → Doctor, Leads_HospitalID → Hospital

Facility(FacilityID, Type, BelongsTo_HospitalID NOT NULL) - FK → Hospital

Hospital(HospitalID, Name)

HasDisease(PatientID, DiseaseID) - genuinely many-to-many, keeps its own table

TreatedBy(PatientID, DoctorID) - genuinely many-to-many, keeps its own table

TreatedAtFacility_ref merged into Patient: Patient(PatientID, Name, TreatedAt_FacilityID NOT NULL)

EmployedBy(DoctorID, HospitalID) - genuinely many-to-many (capped at 3 per doctor by a CHECK/trigger, not by the key), keeps its own table

Only three relationships from Question 1 survive as their own tables: HasDisease, TreatedBy and EmployedBy, because all three are many-to-many. TreatedAt, BelongsTo and Leads were all many-to-one or one-to-one, so each is absorbed as a foreign key: TreatedAt into Patient, BelongsTo into Facility, and Leads into Director (Director already exists as its own table because of the EconomicsDegree attribute, so the foreign key for Leads is simply added there rather than opening a fifth table).

Question 2 - (a) direct transformation (one table per relationship)

Hospital(HospitalID, Name)

Department(HospitalID, DeptNo, Name) - FK HospitalID → Hospital

Employee(EmployeeID, Name)

Doctor(EmployeeID, DoctorsAssocID) - FK → Employee

Nurse(EmployeeID) - FK → Employee

SupportStaff(EmployeeID) - FK → Employee

ChiefDoctor(EmployeeID) - FK → Doctor

HospitalDirector(EmployeeID, EconomicsDegree) - FK → Doctor

Patient(PatientID, Name, AdmissionDate)

Disease(DiseaseID, Name)

ConsistsOf(HospitalID, HospitalID Dept, DeptNo) - redundant given Department already carries HospitalID; not created separately (an identifying relationship is always absorbed into its weak entity, never given its own table)

WorksAtDept(EmployeeID, HospitalID, DeptNo) - FKs → Employee, Department

WorksAtHospital(EmployeeID, HospitalID) - FKs → SupportStaff, Hospital

HasChiefDoctor(EmployeeID, HospitalID, DeptNo) - FKs → ChiefDoctor, Department

LeadsDept(HospitalID, DeptNo, EmployeeID) - FKs → Department, Employee

LeadsHospital(HospitalID, EmployeeID) - FKs → Hospital, HospitalDirector

AdmittedTo(PatientID, HospitalID) - FKs → Patient, Hospital

TreatedIn(PatientID, HospitalID, DeptNo, StartDate, EndDate) - FKs → Patient, Department

TreatedFor(PatientID, DiseaseID, DiagnosisDate) - FKs → Patient, Disease

Note on ConsistsOf: an identifying relationship for a weak entity is never given a separate table under either transformation style - the owner's key is always folded directly into the weak entity's own primary key (Department's key already is HospitalID+DeptNo), so listing it as a fifth 'relationship table' here would just duplicate Department.

Question 2 - (b) minimal transformation

Hospital(HospitalID, Name)

Department(HospitalID, DeptNo, Name, LeadsDept_EmployeeID NOT NULL) - FKs → Hospital, Employee

Employee(EmployeeID, Name, WorksAtDept_HospitalID, WorksAtDept_DeptNo) - FK (composite, nullable together) → Department; null for SupportStaff, not null for Doctor/Nurse

Doctor(EmployeeID, DoctorsAssocID) - FK → Employee

Nurse(EmployeeID) - FK → Employee

SupportStaff(EmployeeID, WorksAtHospital_HospitalID NOT NULL) - FKs → Employee, Hospital

ChiefDoctor(EmployeeID, HasChiefDoctor_HospitalID NOT NULL, HasChiefDoctor_DeptNo NOT NULL) - FKs → Doctor, Department

HospitalDirector(EmployeeID, EconomicsDegree) - FK → Doctor. (LeadsHospital is 1:1 both sides; its foreign key is placed here rather than on Hospital purely as a design choice.)

Patient(PatientID, Name, AdmissionDate, AdmittedTo_HospitalID NOT NULL) - FK → Hospital

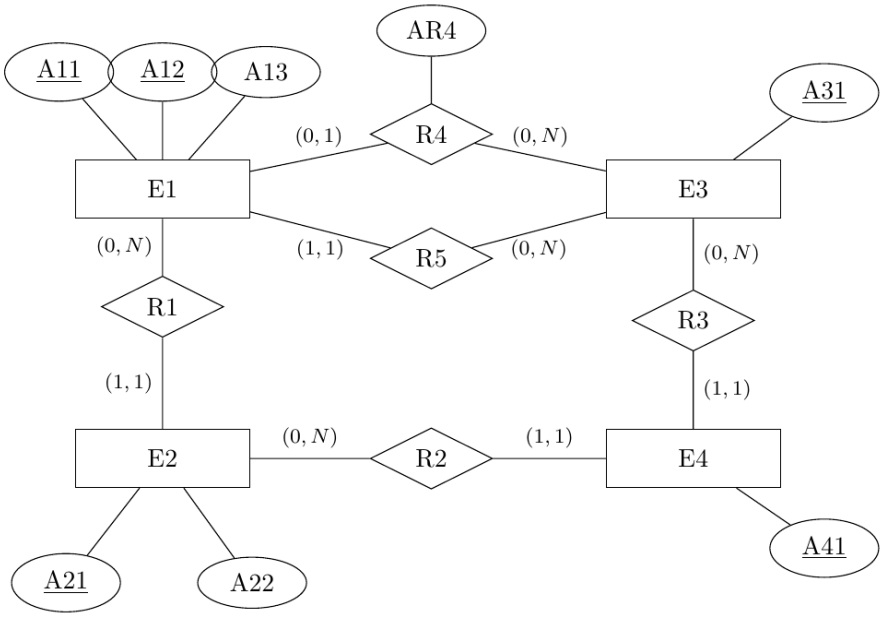
Disease(DiseaseID, Name)

TreatedIn(PatientID, HospitalID, DeptNo, StartDate, EndDate) - genuinely many-to-many, keeps its own table

TreatedFor(PatientID, DiseaseID, DiagnosisDate) - genuinely many-to-many, keeps its own table

One design choice worth stating explicitly: LeadsHospital is one-to-one on both sides (each hospital has exactly one director, each director leads exactly one hospital), so the foreign key could equally well have been placed on Hospital instead of HospitalDirector. Either is correct; the schema above keeps it on HospitalDirector so that Hospital, which is referenced by almost every other table, stays free of a dependency on a subtype three levels down the ISA hierarchy. Only TreatedIn and TreatedFor survive as separate tables from Question 2, again because they are the only genuinely many-to-many relationships in the diagram; every 1:N and 1:1 relationship (ConsistsOf, WorksAtDept, WorksAtHospital, HasChiefDoctor, LeadsDept, LeadsHospital, AdmittedTo) was absorbed as a foreign key instead.

Question 5: Relational Schema with Nullability (Given Diagram)



Given ER diagram, (min,max) notation. Every entity carries its own key: E1 has the composite key (A11, A12); E2, E3 and E4 each have a single-attribute key.

Reading the cardinalities

- **R1**: E1 (0,N) - E2 (1,1). Each E2 relates to exactly one E1; an E1 may relate to many E2's. Many-to-one from E2 to E1.
- **R2**: E2 (0,N) - E4 (1,1). Each E4 relates to exactly one E2. Many-to-one from E4 to E2.
- **R3**: E3 (0,N) - E4 (1,1). Each E4 relates to exactly one E3. Many-to-one from E4 to E3.
- **R4** (carries attribute AR4): E1 (0,1) - E3 (0,N). Each E1 relates to at most one E3, optionally. Many-to-one from E1 to E3, optional on the E1 side.
- **R5**: E1 (1,1) - E3 (0,N). Each E1 relates to exactly one E3, mandatorily. Many-to-one from E1 to E3, mandatory on the E1 side. (A separate relationship from R4, even though it connects the same two entities.)

None of R1-R5 is many-to-many, so, exactly as in Questions 3 and 4(b), every one of them can be folded into the entity on its 'many' side as a foreign key. That gives four relations in total, one per entity, with no separate relationship tables.

Relational schema

`E1(A11, A12, A13, R4_E3, AR4, R5_E3)`

`E2(A21, A22, R1_E1a, R1_E1b)`

`E3(A31)`

`E4(A41, R2_E2, R3_E3)`

Keys and foreign keys

- **E1**: key (A11, A12). Foreign keys R4_E3 → E3(A31) and R5_E3 → E3(A31) - two separate columns, since R4 and R5 are two distinct relationships to the same entity.

- **E2**: key A21. Foreign key (R1_E1a, R1_E1b) → E1(A11, A12), a two-column reference because E1's key is composite.
- **E3**: key A31. No foreign keys - E3 sits on the 'one' side of both R4 and R5, so E1 holds those keys, not E3.
- **E4**: key A41. Foreign keys R2_E2 → E2(A21) and R3_E3 → E3(A31).

Nullability

Every attribute inherited directly from the original ER schema (A11-A13, A21, A22, A31, A41) is NOT NULL, as given. For the columns introduced by folding in a relationship, nullability follows directly from the minimum participation of the entity holding the foreign key:

Relation	Attribute	Nullable?	Reason
E1	A11, A12, A13	NOT NULL	original attributes, given as non-null
E1	R4_E3	NULL allowed	E1's participation in R4 is (0,1) - optional
E1	AR4	NULL allowed	only meaningful when R4_E3 is populated
E1	R5_E3	NOT NULL	E1's participation in R5 is (1,1) - mandatory
E2	A21, A22	NOT NULL	original attributes, given as non-null
E2	R1_E1a, R1_E1b	NOT NULL	E2's participation in R1 is (1,1) - mandatory
E3	A31	NOT NULL	original attribute, given as non-null
E4	A41	NOT NULL	original attribute, given as non-null
E4	R2_E2	NOT NULL	E4's participation in R2 is (1,1) - mandatory
E4	R3_E3	NOT NULL	E4's participation in R3 is (1,1) - mandatory